

Regex-Based Feature Mining, Graph Analytics, and Markov-Chain Simulation of a Cooperative Card Game

A Data Pipeline Built on Arkham Horror: The Card Game

Sergio Soto

Independent research & engineering project — quantitative / data analysis track

Code repository: <https://github.com/justanotheruseeeeeer>

Keywords: feature engineering, regular-expression parsing, natural language n-gram analysis, k-nearest-neighbour graphs, community detection, Markov chains, Monte-Carlo-free stochastic simulation, Bayesian optimization, decision theory.

Abstract

This report documents an end-to-end data-science and simulation pipeline built around *Arkham Horror: The Card Game*, a cooperative deck-building card game. Starting from the public ArkhamDB API, a regex-based parameter miner converts unstructured natural-language card text into a structured numerical schema of roughly twenty gameplay parameters per card, validated against a hand-verified regression suite. On top of this dataset, three complementary analytical layers are developed: (i) a lexical-analysis module that mines unigram, n-gram and skip-gram statistics to surface recurring game-design idioms directly from the corpus, with no predefined keyword list; (ii) a vector-space and graph-analysis module that embeds cards as numerical or lexical feature vectors, builds a k-nearest-neighbour similarity graph, and applies Louvain community detection to recover card archetypes without hand-labeling; and (iii) a directed, weighted graph model — first prototyped for static analysis in Gephi and later re-implemented as an explicit discrete-time Markov chain — that treats a deck as a dynamical system of card, resource, and action nodes, and derives calibrated estimates of expected damage, clue-gathering, and resource output over a full campaign scenario. A closed-form recursive model of the game's chaos-bag success-probability distribution, a Pareto-optimal mulligan decision procedure, and an interactive dashboard complete the toolkit. All components are implemented in Python and evaluated on a real corpus of over 1,200 cards. The project illustrates a complete quantitative research workflow — data acquisition, feature engineering, unsupervised structure discovery, and stochastic-process modelling — applied end-to-end to an unconventional but richly structured real-world dataset.

Resumen

Este informe documenta un pipeline completo de ciencia de datos y simulación construido en torno a *Arkham Horror: El Juego de Cartas*. A partir de la API pública de ArkhamDB, un extractor basado en expresiones regulares convierte el texto no estructurado de cada carta en un esquema numérico de unos veinte parámetros de juego, validado contra una batería de regresión revisada a mano. Sobre este conjunto de datos se desarrollan tres capas analíticas: (i) minería léxica de n-gramas y skip-grams para descubrir patrones de diseño recurrentes sin partir de una lista de palabras clave; (ii) modelado vectorial y de grafos — similitud k-NN y detección de comunidades de Louvain — para descubrir arquetipos de cartas sin etiquetado manual; y (iii) un modelo de grafo dirigido y ponderado, prototipado primero en Gephi y luego reimplementado como una cadena de Markov de tiempo discreto, que trata un mazo como un sistema dinámico calibrado con datos reales de más de 1.200 cartas. Un modelo cerrado y recursivo de la distribución de éxito de la bolsa de caos, un motor de decisión de mulligan Pareto-óptimo, y un dashboard interactivo completan el conjunto de herramientas.

Table of Contents

1. Introduction	5
1.1 Motivation	5
1.2 Arkham Horror: The Card Game — A Rules Primer	5
1.3 Why This Domain Rewards a Data/Quant Approach	5
2. System Architecture Overview	7
3. Data Acquisition and the Card Miner	8
3.1 Source and Type Hierarchy	8
3.2 The Parameter Schema	8
3.3 The Regex Rule Engine	8
3.4 Validation Philosophy	9
4. Lexical Analysis: N-gram Frequency Mining	10
4.1 Tokenization and Two Frequency Regimes	10
4.2 Results on the Full Corpus (1,267 cards with parseable text)	10
4.3 Skip-Grams: Detecting Gapped Patterns	12
5. Vector Space Modeling and Graph Analysis	13
5.1 Two Interchangeable Feature Vectors	13
5.2 Similarity Graph and Community Detection	13
5.3 Results on the Full Library	13
5.4 Interactive Exploration	14
6. Directed Graph Analysis in Gephi: An Agnes Baker Case Study	16
6.1 Node and Edge Construction	16
6.2 Visualizations	16
6.3 Relationship to the Later Simulator	18
7. Deck Construction Pipeline	19
8. Simulation Engine: A Markov Chain Approximation	20
8.1 State Space	20
8.2 Skill Tests: Bridging to the Chaos Bag Model	20
8.3 The Geometric-Clock Approximation for Charges	21
8.4 Resource Scarcity: the "Robbing" Mechanic	21
8.5 Calibration via Bayesian Optimization	21

9. Deck Diagnostics: Spiderweb and Resource-Bottleneck Charts	22
9.1 Spiderweb Chart: Deck Strength Profile	22
9.2 Resource-Bottleneck Heatmap	22
10. Chaos Bag Modifiers: A Closed-Form Model for Bless/Curse	24
10.1 Base Success Probability	24
10.2 Recursive Injection of Bless/Curse Tokens	24
10.3 From Token Count to an Equivalent Duration	25
11. Mulligan Decision Engine	27
11.1 Exact Combinatorial Evaluation, Not Monte Carlo	27
11.2 Pareto-Dominance Filtering	27
11.3 Epsilon-Lexicographic Tie-Breaking	27
12. Interactive Dashboard	28
13. Project Roadmap	29
13.1 Phase Two: Statistical Validation of Deck-Building Hypotheses	30
14. Discussion and Limitations	31
15. Conclusion	31
Appendix A: Repository and Code Structure	33

1. Introduction

1.1 Motivation

Modern trading- and living-card games generate large, richly structured, publicly documented corpora of natural-language rules text with an unusually clean underlying numerical semantics: every card ability can, in principle, be reduced to a small set of quantities — damage dealt, resources gained, cards drawn, probabilities of success. This makes card games an attractive sandbox for a full quantitative-research pipeline: text mining and feature engineering to turn language into numbers, unsupervised learning to discover structure without hand-labeling, and stochastic-process modelling to turn a static card pool into a dynamic prediction of play. Unlike many toy datasets used for portfolio projects, the corpus here is genuinely irregular, ambiguous, and adversarial to naive parsing — which is precisely why it is a useful testbed for the kind of careful, iteratively-debugged rule engineering that real-world messy data demands.

1.2 Arkham Horror: The Card Game — A Rules Primer

Arkham Horror: The Card Game (Fantasy Flight Games) is a cooperative, Lovecraft-themed living card game (LCG) for one to four players. Each player controls an **investigator** with fixed base statistics across four skills — **willpower**, **intellect**, **combat**, and **agility** — and builds a personal 30-card deck from cards belonging to one or more of five overlapping classes (Guardian, Seeker, Rogue, Mystic, Survivor), subject to deck-size, faction-affinity and experience-point (XP) constraints. Cards are one of three basic types: **assets** (persistent permanents that occupy play, e.g. weapons, allies, spells), **events** (one-shot effects, discarded after use), and **skill cards** (played face-down onto a test as a committed bonus, revealed only if needed).

Play proceeds in scripted **turns** subdivided into a Mythos phase (the shared encounter deck advances the scenario and can attack investigators or add doom to the game), an Investigation phase (each investigator takes exactly **three actions**, drawn from a fixed action economy: playing cards, investigating a location for clues, attacking or evading enemies, resting, moving, or activating card abilities), an Enemy phase (unengaged hostile enemies attack), and an Upkeep phase (untapping/refreshing resources and drawing a card). The central uncertainty mechanic is the **skill test**: an investigator commits a base skill value plus any icons contributed by committed cards, then draws one token from an opaque **chaos bag** containing numeric modifiers (and occasionally special symbols) to determine success or failure against a target difficulty. The chaos bag's contents are themselves modifiable in play through the **bless/curse** mechanic, which is treated formally in Section 10.

This rules substrate is what gives the corpus its numerical semantics: a card's textual ability almost always reduces to a combination of (a) a fixed or conditional numeric payload (damage, clues, resources, cards drawn), (b) a cost in actions and/or resources, (c) a probability of firing (deterministic, or gated behind a skill test against the chaos bag), and (d) a repetition budget (a single use, a limited number of "charges," or an unlimited exhaust/ready cycle). The entire project can be read as the construction of a parser and simulator for exactly this four-tuple, at the scale of the full published card pool.

1.3 Why This Domain Rewards a Data/Quant Approach

- The raw data (card text) is genuinely unstructured and adversarial: near-identical phrasings can carry different arithmetic depending on card type, and a naive keyword search systematically over- or under-counts effects — a realistic proxy for messy production NLP problems.
- The domain admits ground truth: a human expert (the author) can hand-verify a regression suite of known-tricky cards, giving a genuine train/validate loop for the rule engine, much as one would validate a production ETL pipeline against a golden dataset.

- The resulting structured dataset supports every standard layer of a quantitative pipeline: lexical statistics, vector-space embeddings, graph theory, Markov-chain simulation, closed-form probability modelling, and multi-criteria decision theory — all demonstrated in this single report on one coherent dataset.

2. System Architecture Overview

The project is organized as a linear data-acquisition stage feeding three parallel analytical branches, which in turn feed a simulation and decision layer. Figure 1 summarizes the full pipeline; each block is described in its own section below.

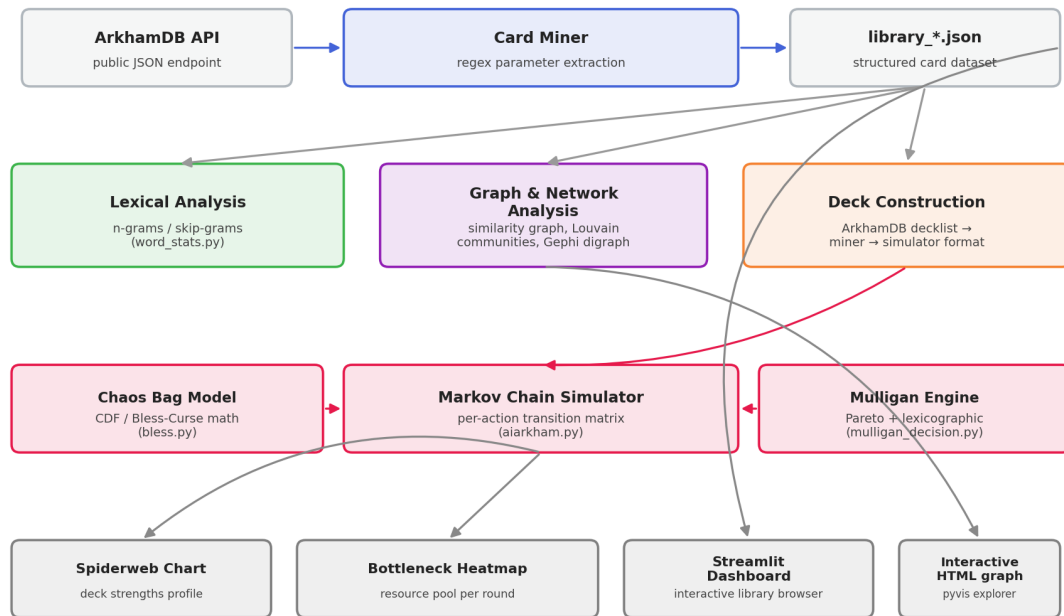


Figure: End-to-end data pipeline, from raw card text to statistical, network, and simulation outputs.

Figure 1. End-to-end pipeline: ArkhamDB API -> regex miner -> structured JSON dataset, feeding lexical analysis, graph analysis, and deck construction, which in turn feed the Markov-chain simulator (informed by the chaos-bag model and the mulligan engine) and its diagnostic outputs.

3. Data Acquisition and the Card Miner

3.1 Source and Type Hierarchy

Raw card data is retrieved from ArkhamDB's public REST endpoint (<https://arkhamdb.com/api/public/cards/>), which returns, per card, free-text ability text alongside structured metadata (cost, traits, faction, base stats, health/sanity, XP level). Every card is classified into one of five mutually exclusive types before any numeric extraction takes place:

Type	Definition
skill	Raw skill card, committed face-down to a test.
pseudo_skill	A fast event that triggers <i>on a test result</i> (e.g. "Lucky!", "Oops!"). Action cost is always zero; its conditions are assumed fulfilled.
event	A standard one-shot event card.
asset	A standard persistent asset.
pseudo_asset	An event that attaches to something or carries its own charge pool (modelled identically to an asset).

3.2 The Parameter Schema

Every card is reduced to a fixed-width numerical record (the Parameters dictionary) capturing its gameplay payload independently of its natural-language phrasing:

Key	Meaning
dm / dm_react	Active damage per use / reaction damage.
inv	Clues generated per use (1 = investigate action, N = fixed discovery).
evade	1 if the card evades; combined with fix to distinguish automatic vs. test-gated.
draw / search / res	Cards drawn / cards searched / resources gained.
heal_dmg / heal_hor	Damage / horror healed.
soak_dmg / soak_hor	Passive soak from an asset's own health/sanity.
fix	1 if the effect fires unconditionally, without a skill test.
charges / charge_cost	Size of the use-pool / per-activation cost.
exhaust_use	True if the card exhausts as its cost (divides its effect by 3, see §3.3).
add / add_stat	Literal +N bonus to a skill value / which stat is added to another.
red / duration	Shroud or difficulty reduction / how many tests it lasts.
AcP	Action-economy score (see §3.3).
stat_wil/int/com/agi	Permanent stat bonus granted by an asset's own header.

3.3 The Regex Rule Engine

Parameter extraction is implemented as a cascade of context-sensitive regular expressions rather than a single flat keyword search, because the same surface phrasing carries different arithmetic depending on card type. The canonical example, worth stating explicitly because it recurs throughout the design: the phrase "*this attack deals +1 damage*" means a total of 2 damage on an asset or event (a base fight action, worth 1, plus the stated bonus), but only 1 damage of marginal value on a *skill* card, because on a skill the fight action itself is the test the card is committed to — the card must report only the incremental value it contributes, not the base action it is riding on. This correction is applied post-extraction, keyed purely on the already-known `card_type`, never inferred from the text itself:

```

if is_skill_type and e['dm'] > 0 and _dm_is_modifier:
    e['dm'] = e['dm'] - 1 # the skill only contributes its bonus
    e['fix'] = 0         # its outcome now depends on the test

```

The action-economy parameter (AcP) follows a similar small rule table, reproduced here because it is one of the more subtle and heavily contested design decisions in the engine:

Rule	AcP contribution
Card is Fast on play (any type)	+1
[fast] ability, non-exhaust, on an asset	+1
[fast] or [reaction] ability, exhaust, on an asset	+0.33 (fires ~once per 3 actions)
Connecting-location move as a bonus on top of another effect	+1
Sole-purpose exhaust move (the whole ability IS the move)	+0 before the exhaust ÷3
Explicit additional actions granted	+N (÷3 if exhaust)
pseudo_skill	0, always — enforced at the end of extraction

Charge accounting follows the convention that an explicit uses(N) clause is authoritative; otherwise a non-exhaust asset defaults to 7 charges and an exhaust asset to 21 (7×3 , reflecting that its per-use effect is already divided by 3), while events, skills, and pseudo-skills default to a single use.

3.4 Validation Philosophy

The engine is validated the way a production ETL pipeline would be: a hand-verified regression suite of known-tricky cards (asymmetric skill corrections, multi-paragraph weighted averages, discard-cost abilities that must be ignored, "succeed by N" conditional bonuses, stat-substitution vs. stat-addition phrasing) is re-run after every change to the rule set, and the full library is periodically re-mined and spot-checked against a set of suspicion heuristics (e.g. flag any card with zero damage despite containing a fight/attack keyword, or any asset with more than 21 charges). This regression-first workflow is what allowed the rule set to be iteratively hardened against real edge cases — conditional damage that averages over two outcomes, reaction abilities nested inside multi-line trigger text, and play-condition headers that must be excluded from effect extraction — without silently reintroducing previously-fixed bugs.

4. Lexical Analysis: N-gram Frequency Mining

Independently of the hand-written extraction rules, a purely data-driven lexical layer answers a different question: *which words and phrases actually recur across the corpus, and how widely?* This module requires no predefined keyword list — it discovers the vocabulary from the text itself, and its output both cross-validates the manually written regex rules and serves as the raw material for the vector-space model in Section 5.

4.1 Tokenization and Two Frequency Regimes

Text is lowercased, bracketed game icons ([willpower], [reaction], ...) are preserved as single tokens, and every numeral is normalized to a placeholder token so that "discover 1 clue" and "discover 3 clues" count as the same underlying expression. For single words, common English function words ("the", "a", "of", ...) are removed, since otherwise they dominate the ranking without carrying any game-design signal. For multi-word phrases, function words are *kept* — removing them would destroy meaningful expressions such as "you may move" — and only n-grams that are *purely* function words (e.g. "of the") are discarded as pure grammatical noise. Every term is reported with two distinct statistics: raw frequency (total occurrences, which a single verbose card can inflate) and **document frequency** (the number of distinct cards in which the term appears at least once) — the latter is normally the correct proxy for how "important" or widespread a mechanic really is.

4.2 Results on the Full Corpus (1,267 cards with parseable text)

Table 4 shows the fifteen most widespread single words in the corpus, ranked by document frequency.

#	Unigram	freq	df
1	location	753	355
2	play	420	265
3	test	579	248
4	skill	490	239
5	damage	481	234
6	enemy	573	233
7	exhaust	316	218
8	card	392	200
9	[reaction]	267	194
10	investigator	349	190
11	spend	292	187
12	uses	305	179
13	[action]	354	173
14	discard	233	158
15	hand	238	154

Table 5 shows the fifteen most widespread two-word expressions.

#	Bigram	freq	df
1	your location	473	291
2	<n> damage	359	199
3	skill test	321	182
4	spend <n>	260	165
5	uses <n>	258	163

#	Bigram	freq	df
6	you succeed	235	133
7	fast play	188	126
8	an enemy	178	121
9	this attack	361	119
10	b fight	195	116
11	fight b	194	115
12	your hand	154	110
13	a skill	165	106
14	deal <n>	170	106
15	[reaction] when	127	101

Table 6 lists the twelve most widespread three-word expressions, and Table 7 the twelve most widespread four-word expressions — longer n-grams recover complete templated ability clauses ("investigator at your location", "attack deals <n> damage") rather than isolated keywords.

#	Trigram	freq	df
1	at your location	377	243
2	if you succeed	205	116
3	b fight b	193	115
4	a skill test	159	105
5	for this attack	184	103
6	deal <n> damage	160	101
7	get <n> [combat]	132	81
8	investigator at your	109	76
9	enemy at your	95	71
10	an investigator at	93	64
11	draw <n> card	89	63
12	this skill test	94	63

#	4-gram	freq	df
1	you get <n> [combat]	129	79
2	investigator at your location	109	76
3	enemy at your location	95	71
4	an investigator at your	90	61
5	b you get <n>	91	55
6	clue at your location	73	55
7	get <n> skill value	69	53
8	b fight b you	87	52
9	deal <n> damage to	79	50
10	attack deals <n> damage	85	50
11	skill value for this	75	50
12	[combat] for this attack	75	49

4.3 Skip-Grams: Detecting Gapped Patterns

Contiguous n-grams cannot detect a pattern such as *"if you succeed by ... this attack deals ..."*, because the gap between "succeed by" and "deals" varies in length from card to card — a fixed-width window will match some cards and silently miss others. The engine additionally computes **skip-grams**: pairs of content words that co-occur within a sliding window of N tokens, irrespective of what lies between them or how far apart they are on different cards. This is the same principle behind classical collocation statistics in corpus linguistics, adapted here to surface exactly the conditional-effect templates that motivated the "succeed-by" correction logic in the miner (Section 3.3): once a candidate gapped pattern is found by this exploratory pass, it can be promoted to an explicit, precisely-scoped regular expression in the rule engine, closing the loop between data-driven discovery and hand-engineered extraction.

5. Vector Space Modeling and Graph Analysis

Both the numerical parameter schema (Section 3) and the lexical vocabulary (Section 4) can serve as the coordinate system for a card embedding. This section builds a similarity graph over the card pool and recovers archetypes as emergent graph communities, deliberately avoiding any hand-specified list of "card types" as the grouping criterion.

5.1 Two Interchangeable Feature Vectors

The graph-construction code is agnostic to how a card is embedded: any function `card -> vector` can be plugged in without touching the rest of the pipeline. Two vectorizers are implemented:

- **Numerical vectorizer:** each card becomes a vector of its `Parameters` fields (damage, clues, resources, charges, ACP, permanent stat bonuses, cost, and icon pips), standardized to zero mean and unit variance per feature before computing similarity — without this step, a large-range field such as charges (up to 21) would dominate a small-range field such as damage (0–4) purely because of its numerical scale, not its actual importance.
- **Lexical (bag-of-expressions) vectorizer:** each card becomes a binary vector over the ~100 most document-frequent n-grams discovered in Section 4, with a 1 in position i if that expression appears anywhere in the card's text. This lets the grouping emerge directly from shared phrasing in the corpus rather than from a parameter list chosen in advance by the author.

5.2 Similarity Graph and Community Detection

Given a vectorizer, pairwise **cosine similarity** is computed between every pair of cards, and each card is connected to its k most similar neighbours (k-nearest-neighbour graph), which keeps the graph sparse and legible — an all-pairs complete graph on a corpus this size would be uninformative. An explicit minimum-similarity threshold discards edges between near-zero vectors (cards with almost no comparable parameters, which would otherwise be connected essentially at random, since their cosine similarity to everything is approximately equal). **Louvain community detection** — a standard modularity-maximization algorithm for graph clustering — is then applied to recover groups of mutually similar cards. Crucially, these communities are *discovered*, not defined: the algorithm is given no information about card type or intended archetype, only the similarity graph itself.

5.3 Results on the Full Library

Applied to the full corpus, the k-nearest-neighbour graph ($k=3$, numerical vectorizer) collapses 1,276 raw entries to 858 uniquely-named nodes — a useful data-quality observation in itself: the ArkhamDB export contains multiple printings and experience-level variants sharing an identical display name, which merge into a single graph node under a name-keyed identity. Louvain detection recovers 29 communities. Figure 2 shows the resulting layout, using a force-directed (spring) layout in which connected nodes attract and all node pairs repel, so that densely inter-similar clusters visually separate from the rest of the pool; only the highest-degree ("hub") card in each dense region is labelled, to keep the figure legible at this scale.

Grafo de similitud de cartas (29 comunidades, 858 cartas)

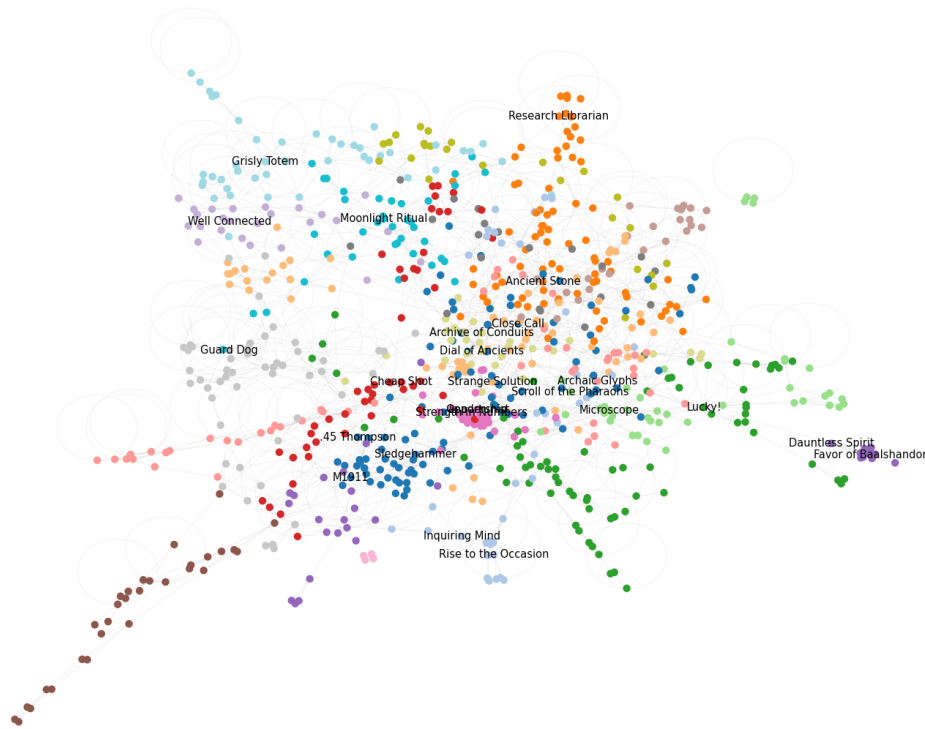


Figure 2. k -NN similarity graph over 858 uniquely-named cards from the mystic-faction-plus-neutral library, coloured by Louvain community (29 communities). Layout: force-directed spring embedding; only high-degree hub nodes are labelled.

Manual inspection of individual communities confirms the graph recovers gameplay-meaningful groupings without being told what to look for: e.g. a tight cluster around *Research Librarian* aggregates card-advantage and search effects, while a cluster around *.45 Thompson* and *Stoic Fighter*—style cards aggregates raw combat-damage assets.

5.4 Interactive Exploration

A companion interactive version of the same graph is generated with a physics-based JavaScript network renderer (pyvis/vis-network): every card is a plain point with no visible label, and the name only appears as a tooltip on hover, while the layout can be freely dragged, zoomed, and panned. This is the practically usable exploration tool for a corpus of this size; the static figure above is reproducible from the same underlying graph object purely for print/PDF purposes.

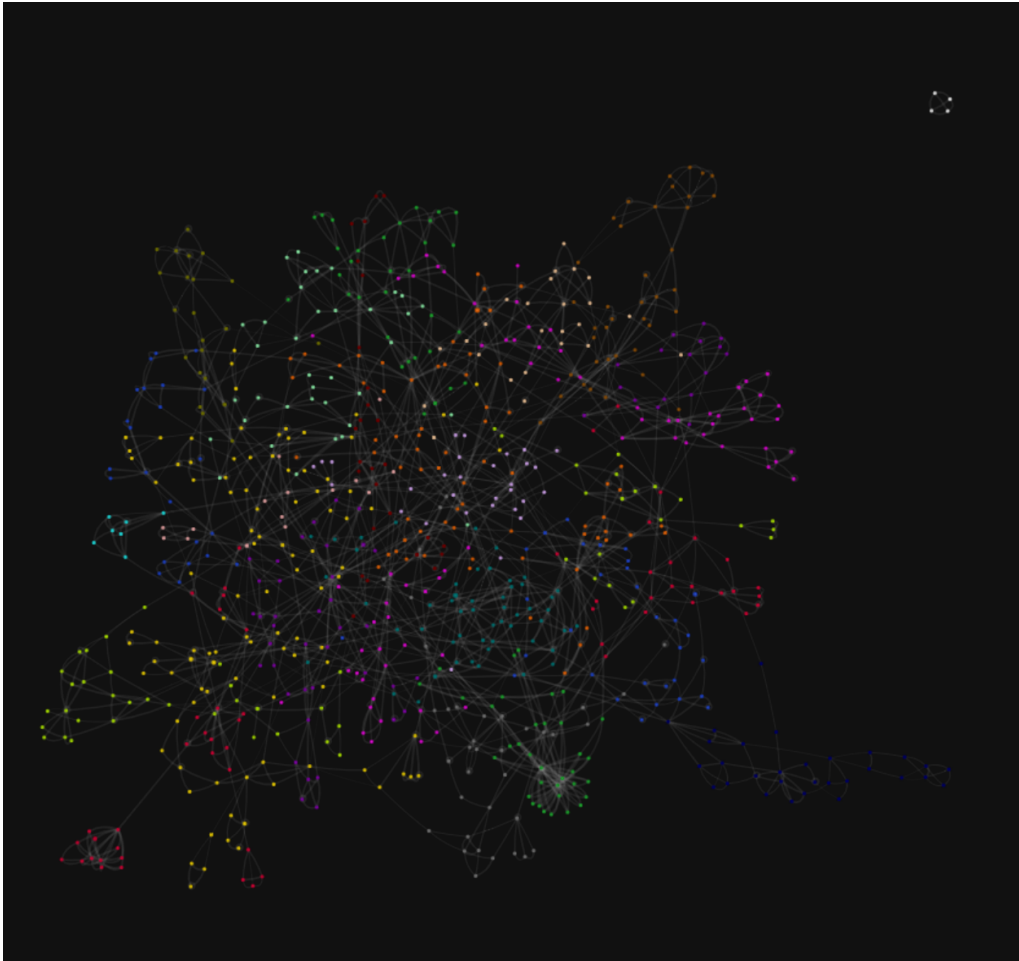


Figure 3. Screenshot of the interactive pyvis rendering of the full similarity graph. Node colour encodes Louvain community; node identity is revealed only on hover, keeping the canvas legible at full corpus scale.

6. Directed Graph Analysis in Gephi: An Agnes Baker Case Study

Before the Markov-chain simulator of Section 8 was developed, the same underlying idea — a deck as a network of cards and effect categories — was first prototyped as a static **directed, weighted graph**, exported to CSV and explored visually in [Gephi](#), the open-source network-analysis and visualization tool. This earlier prototype (built for a specific 33-card investigator deck around the investigator *Agnes Baker*) is architecturally the ancestor of the simulator, and is kept in the toolkit because a static directed-graph view answers a genuinely different question than a Markov chain does: *not* "what is the expected outcome over N turns", but "which nodes act as structural hubs of the deck's economy, and how does influence flow between cards and effect categories, independent of any time horizon?"

6.1 Node and Edge Construction

Every card in the deck becomes a node. Alongside them, a fixed set of **metagame sink nodes** represents the effect categories a card's output can flow into (Cost, Damage, Investigate, Evade, Resourceful, and healing), and a small ring of **action-clock nodes** (Action, Action1, Action2) represents the cyclical consumption of the three actions available each turn. A directed edge from a card to a metagame node is weighted by how much of that category the card produces per use; an edge from each metagame/action node back into every card is weighted by $1/\text{deck_size}$, approximating the probability of drawing any given card at a random point in the game. Repeatable effects are additionally modelled as self-loops on the card's own node — the same "repetition budget" concept later formalized as the geometric charge-clock of Section 8.4.

6.2 Visualizations

Figure 4 uses a radial ("sphere") layout algorithm, which naturally foregrounds the metagame hub nodes (Cost, Damage, Investigate, Evade, Resourceful) at the visual centre, since they are by construction the highest in-degree nodes in the graph — nearly every card connects to at least one of them. Edge thickness is proportional to weight, so the thick dark bundle collapsing into "Damage" immediately visually identifies this particular Agnes Baker build as damage-centric.

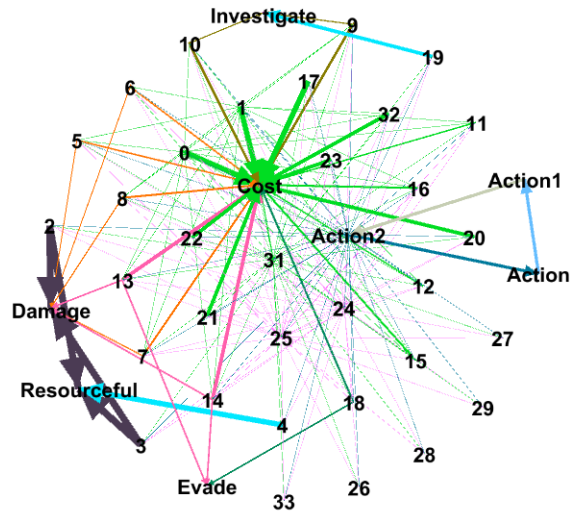


Figure 4. Radial ("sphere") Gephi layout of the Agnes Baker deck graph. Metagame sink nodes (Cost, Damage, Investigate, Evade, Resourceful, Action/Action1/Action2) sit at the structural centre by construction, since almost every card node connects into them.

Figure 5 re-renders the identical graph with a force-directed layout instead, which trades the star-like hub-centric legibility of the radial layout for a clearer view of secondary structure: which individual cards cluster near each other because they share several metagame edges in common, versus which cards are structurally peripheral (thin, isolated connections) to the deck's core economy.

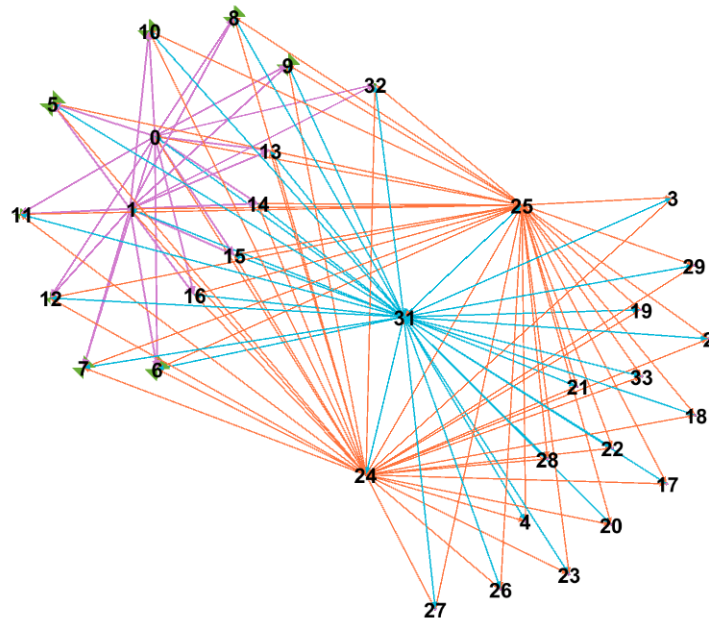


Figure 5. Force-directed Gephi layout of the same directed graph as Figure 4, numbered by node index. A small number of high in-degree metagame nodes visibly organize the rest of the card population into distinct clusters around them.

6.3 Relationship to the Later Simulator

This Gephi prototype fixes the number of Markov-chain steps at a constant 45 actions (15 turns) and computes a single matrix power rather than iterating action-by-action; it therefore cannot express turn-dependent effects (resources accrue at a constant rate every turn regardless of what has already been drawn) or the resource-scarcity "bottleneck" dynamic of Section 9. Section 8 re-implements the same node/edge philosophy as an explicit class-based simulator with a step-by-step transition update, a calibrated action-economy parameter, and the additional resource-robbing mechanic — the directed-graph view here should be read as the static, structural counterpart to the fully dynamic model developed later.

7. Deck Construction Pipeline

The miner's output schema (Section 3.2) and the simulator's expected input schema evolved independently and use different field names for the same underlying concepts — a common situation in any pipeline where an ETL stage and a downstream consumer are built at different times. Rather than rewrite either side, a dedicated **translation layer** converts an ArkhamDB decklist end-to-end into simulator-ready records:

- Fetch a public decklist by ID from ArkhamDB's decklist API and expand its `s l o t s` (card code -> quantity) into an explicit list of card codes.
- Join each code against the locally mined library to recover the full mined record, reporting any codes that fail to resolve (e.g. cards outside the mined faction/cycle).
- Re-map field names: the miner's confusingly-named `I d` field (actually the card's display name) becomes `name`; a list-valued `G o a l _ T y p e` collapses to the single most specific simulator category; `charges` is renamed `num_sea` for search-granting cards and `t i m e` otherwise — the same underlying quantity (a repetition budget) surfaces under two different names depending on card context, a naming decision made early in the simulator's development and preserved here for compatibility rather than re-litigated.
- Cards whose type resolves to a pure skill card have their effect parameters (damage, clues, etc.) stripped and replaced with a single aggregate icon count, matching how the simulator treats committed skill cards as pure test-modifiers rather than autonomous effects.

The translator is implemented as a small self-contained class so it can be driven from the command line with nothing but a deck ID and a path to the local library file, decoupling "which deck am I analysing today" from the rest of the codebase.

8. Simulation Engine: A Markov Chain Approximation

The central simulation question is: given a fixed 30-card deck, what is the expected damage, clue-gathering, resource generation, and survivability over the course of a scenario? A full Monte-Carlo simulation (repeatedly shuffling, drawing, and playing out thousands of games) is the obvious brute-force answer, but it is slow to iterate on during deck design and awkward to differentiate or optimize against. The engine instead models the deck as an explicit **discrete-time Markov chain**: a probability vector over states, updated by repeated multiplication against a transition matrix, one action at a time. This trades some realism for closed-form speed and, more importantly, for a model that can be directly calibrated and queried analytically rather than only sampled.

8.1 State Space

The state space is the union of three node families, mirroring the structure already introduced in the Gephi prototype of Section 6, but now driving an explicit iterative update rather than a single fixed-power matrix multiplication:

- **Card nodes** — one per deck slot, representing "this card is currently the one drawn and available to act with".
- **Metagame sink nodes** — Damage, Investigate, Evade, Resourceful, healing — absorbing state representing which category of effect fired.
- **Action-clock nodes** (Action, Action1, Action2) — a cyclical sub-chain that tracks which of the turn's three actions is currently being spent, so that action-cost bookkeeping is part of the state itself rather than an external counter.

Transition weights out of a card node are set from that card's mined `Parameters` (Section 3.2): a fixed fraction of its outgoing weight flows to each metagame sink it can produce, proportional to its expected output; a calibration parameter *alpha* (see §8.7) governs the split between spending an action on a skill/support card versus a direct-effect card, and is tuned rather than hand-set.

8.2 Skill Tests: Bridging to the Chaos Bag Model

Whenever a transition is gated behind a skill test (e.g. an attack that requires beating the location's shroud value), its effective weight is not a fixed constant but is computed by the same closed-form chaos-bag success-probability function developed independently in Section 10 — the two components of the toolkit share this function rather than duplicating the probability logic, so a change to the chaos-bag composition (e.g. more curse tokens sealed into the bag) automatically propagates into the simulator's transition weights.

8.3 The Geometric-Clock Approximation for Charges

This is the single deliberate departure from strict realism in the entire simulator, and it is worth stating plainly rather than glossing over. A card with, say, 4 charges should fire exactly 4 times and then stop — a hard countdown. A hard countdown is not compatible with the Markov property: the probability of the card still being usable on the next action would depend on how many times it has already fired, i.e. on history, not merely on the current state. To keep the whole system a clean, closed-form Markov chain, each charge-limited card is instead given a **self-loop with a geometric decay**: the per-activation probability of the card "surviving" to be used again is chosen so that its expected number of activations before exhausting, taken to infinity, matches its true charge count. This is exact in expectation but wrong in shape — a geometric distribution has positive probability of firing far more or far fewer times than the true fixed count, whereas the real card cannot exceed its stated charges under any circumstance. The approximation error this introduces is systematically diluted, not eliminated, by calibrating the total number of simulated turns to a realistic scenario length: over a short enough horizon the geometric tail contributes negligible extra mass beyond the true charge count, so the practical divergence between the two models stays small even though it is not zero. This is flagged here explicitly as the model's principal known limitation (see also Section 14).

8.4 Resource Scarcity: the "Robbing" Mechanic

Investigators cannot simply play every card they draw — resources are scarce and regenerate slowly (one per turn at upkeep, plus whatever cards themselves generate). When the simulated state calls for spending more resources than are currently available, the model "robs" the shortfall: it records the deficit and effectively treats that action as consumed without full payoff, rather than allowing free, unconstrained spending. Accumulating these deficits per action and grouping them into rounds of three actions is exactly what produces the resource-bottleneck diagnostic of Section 9.2 — a direct, visual answer to "which stretch of the game is this deck starved for resources?"

8.5 Calibration via Bayesian Optimization

The *alpha* parameter that governs how often the simulated player prioritizes committing a skill card over taking a direct-effect action is not hand-tuned; it is fit with Bayesian optimization (sequential model-based global optimization over a black-box objective), searching for the value that maximizes an overall deck-fitness score. This is a natural fit for this calibration problem: the objective function requires running the full simulator end-to-end for every candidate value, so gradient information is unavailable and each evaluation is comparatively expensive, exactly the regime Bayesian optimization is designed for, in contrast to a dense grid search.

9. Deck Diagnostics: Spiderweb and Resource-Bottleneck Charts

The simulator resolves to two complementary diagnostic visualizations, both generated directly from a run of the simulator on a real example deck. Both are designed as templates: the same two plotting functions accept any deck/chaos-bag pair, so swapping in a different decklist or a different chaos-bag composition regenerates both figures without any change to the plotting code itself.

9.1 Spiderweb Chart: Deck Strength Profile

The simulator's terminal state distribution is aggregated by metagame category (Damage, Investigate, Evade, Resourceful, healing, and an overall durability score) and rendered as a radar / spiderweb chart, giving an immediate visual signature of what a deck is built to do. Figure 6 shows this chart for the example configuration bundled with the simulator: a build strongly weighted toward Damage, with a secondary Resourceful lobe and comparatively little investment in clue-gathering or healing.

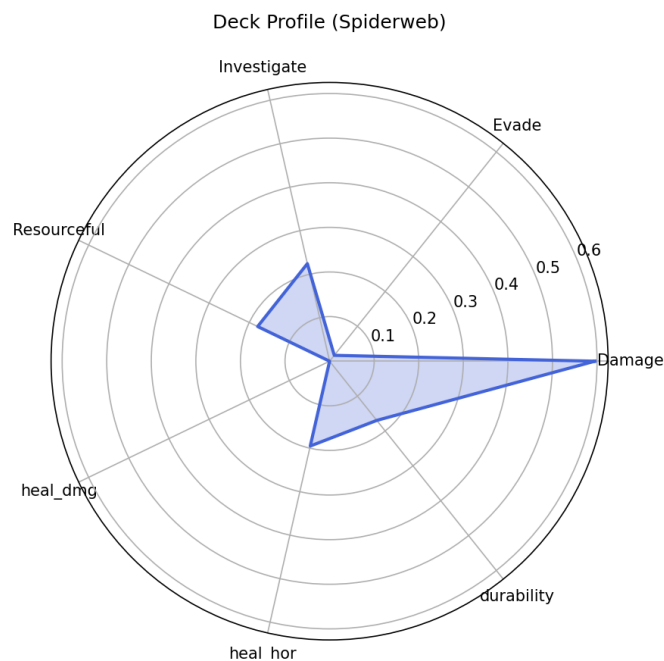


Figure 6. Deck strength profile (spiderweb chart) generated directly from the simulator for the bundled example deck. Each axis is a metagame category; the shaded area is the deck's expected per-action output share in that category.

9.2 Resource-Bottleneck Heatmap

The resource deficits recorded by the "robbing" mechanic of Section 8.6 are grouped into rounds of three actions (one full player-turn) and rendered as a one-dimensional heatmap across the scenario's duration. Figure 7 shows the same example deck: an early-game and a mid-game stretch of resource strain (lighter/warmer cells), separated by a comparatively comfortable middle stretch once the deck's economy cards have had time to come online — precisely the kind of curve-shape question a deck-builder wants answered before physically playtesting a build over several sessions.

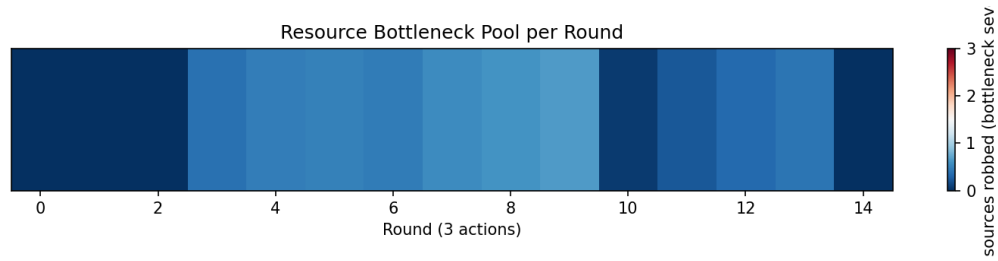


Figure 7. Resource-bottleneck heatmap for the same example deck: darker/cooler cells indicate rounds with no resource shortfall, lighter/warmer cells indicate rounds where the simulated player was forced to "rob" (defer or skip) an action for lack of resources.

10. Chaos Bag Modifiers: A Closed-Form Model for Bless/Curse

The base chaos bag is a small, fixed multiset of numeric token values (fourteen tokens for a standard scenario, ranging from +1 to -4 in this implementation). **Bless** and **Curse** tokens are a chaos-bag-manipulation mechanic that inserts additional, temporary tokens (worth a flat +2 and -2 respectively) into the bag, shifting every subsequent draw's distribution in the corresponding direction until they are drawn and removed. Rather than brute-force enumerate every possible bag permutation as bless/curse tokens accumulate, this module derives a closed-form, recursive success-probability model.

10.1 Base Success Probability

The base bag is first converted into a smoothed empirical CDF, $v(x)$, via linear interpolation between integer token values (allowing fractional test modifiers to be evaluated continuously rather than only at the bag's native integer granularity). From it, $G(x)$ gives the probability of success for a test with net modifier x (skill value plus committed bonuses minus the location's difficulty), and $F(x, y)$ wraps it with the game's rule that a non-positive difficulty is an automatic success.

10.2 Recursive Injection of Bless/Curse Tokens

Let $CDF(x, b, c)$ denote the success probability at modifier x with b bless and c curse tokens currently mixed into the bag. On the next draw, exactly one of three things happens: a base-bag token is drawn (probability proportional to the base bag's size), a bless token is drawn (probability proportional to b , after which the test proceeds as if shifted by +2 with one fewer bless token remaining), or a curse token is drawn (symmetric, -2 shift, one fewer curse token). This gives the exact recursion

$$CDF(x, b, c) = G(x) \cdot (n / (n+b+c)) \quad \# \text{ base token drawn} \\ + CDF(x+2, b-1, c) \cdot (b / (n+b+c)) \quad \# \text{ a Bless token drawn} \\ + CDF(x-2, b, c-1) \cdot (c / (n+b+c)) \quad \# \text{ a Curse token drawn}$$

with n the fixed base-bag size, memoized to avoid recomputation across the shared sub-problems that naturally arise as the recursion unwinds. Figure 8 sweeps this recursion over a 3×3 grid of (bless, curse) combinations, overlaying each curve on the unmodified baseline; the per-panel "Add" figure is a derived summary statistic that maps a given bless/curse mixture onto an equivalent flat modifier value, expressed in the same units as the add/duration parameters already present in the card schema (Section 3.2) — the explicit design goal being to make a chaos-bag-manipulation card and a flat-bonus card comparable on a single numeric scale, rather than requiring the downstream simulator to treat them as two unrelated mechanics.

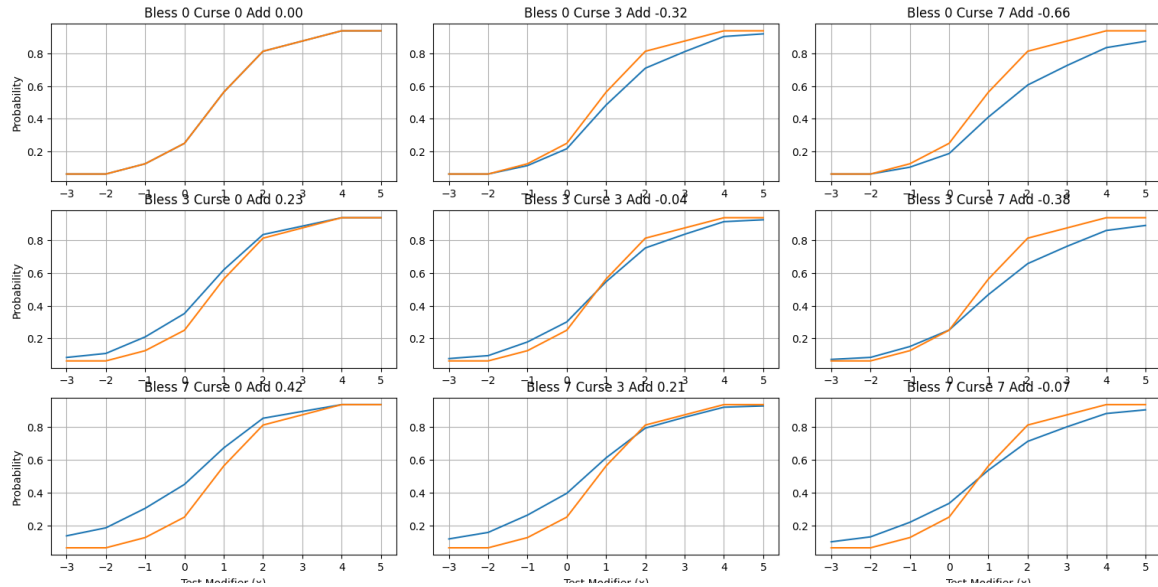


Figure 8. Success-probability curves under different (Bless, Curse) token counts (rows/columns), each overlaid against the unmodified baseline (orange/blue traces) with the derived equivalent flat modifier annotated in each panel title.

10.3 From Token Count to an Equivalent Duration

A companion recursion, $\text{TCDF}(b)$, computes something more specific: the **expected number of skill tests** that must be taken before all b accumulated Bless (or Curse) tokens have been drawn out of the bag — these tokens, unlike ordinary chaos tokens, are permanently removed once drawn rather than returned, so this is a genuine expected waiting time until exhaustion, not a steady-state property of the bag. This quantity turns out to be a good proxy for duration in the card schema of Section 3.2: rather than reasoning about a bless/curse effect as an intractable shifting distribution, it can be approximated as an equivalent flat modifier applied for a fixed number of tests, i.e. $\text{total_modifier} \sim (\text{bless} - \text{curse}) \times 2 / \text{duration}$, with duration given by this expected-tests-to-exhaustion value — folding the Bless/Curse mechanic into exactly the same add/duration representation already used for ordinary flat-bonus cards, rather than requiring the downstream simulator to special-case it. Figure 9 shows this expected duration growing **logarithmically** with the number of accumulated tokens: each additional token buys a progressively smaller increase in expected tests-to-exhaustion, since it is competing against an ever-larger effective bag. The author's own validation note on this result (retained here for transparency) observes that the derived value tracks a simpler linear heuristic of $(\text{bless} - 2 \times \text{curse}) / \text{turns}$ reasonably closely in most regimes, but diverges — landing between $1.5\times$ and $2\times$ the naive estimate — once curse tokens numerically dominate bless tokens, a useful caveat for anyone tempted to substitute the cheaper linear approximation without checking which regime they are in.

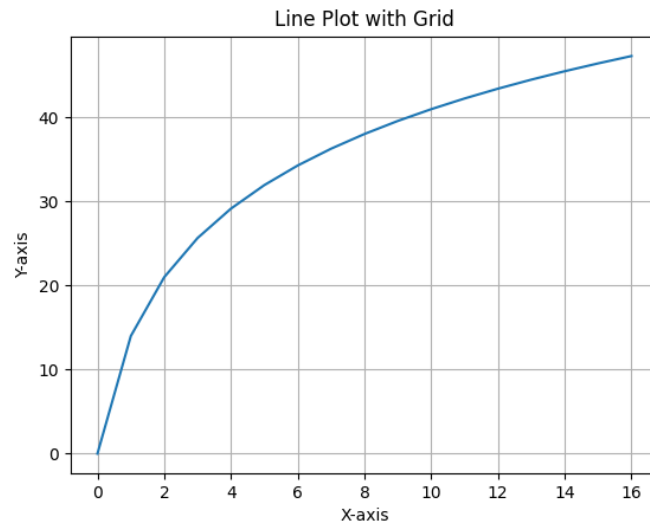


Figure 9. Expected number of skill tests until all accumulated Bless tokens are drawn out of the bag, as a function of token count — the logarithmic, saturating shape is the basis for the duration estimate used in Section 10.3.

11. Mulligan Decision Engine

At the start of a scenario, a player may discard and redraw any subset of their opening hand exactly once (a "mulligan"). Deciding how many cards to mulligan, given a per-card quality score, is a small but genuine decision-theory problem: it is tempting to simply maximize expected value, but expected value is a fragile criterion when the underlying quality score is better thought of as an ordinal ranking than as a cardinal, correctly-scaled utility.

11.1 Exact Combinatorial Evaluation, Not Monte Carlo

For each candidate mulligan count k , the module computes the *exact* distribution of outcomes by enumerating every possible k -card combination drawable from the remaining deck (`itertools.combinations`), rather than approximating it by repeated random sampling. This is deliberate: opening-hand-style problems have a modest, enumerable combination space, so an exact combinatorial pass is both cheaper and free of sampling noise compared to running thousands of simulated mulligans, in contrast to the full game simulator of Section 8, where the state space is far too large for exact enumeration and a Markov-chain relaxation is the more appropriate tool.

From this exact distribution, two heuristics are reported for every candidate k : the expected value **EV** of the resulting hand, and **P(improvement)**, the probability that mulliganing k cards yields a strictly better hand than keeping the original one.

11.2 Pareto-Dominance Filtering

Rather than combine EV and P(improvement) into a single weighted score — which would require committing to a specific, hard-to-justify trade-off rate between the two — candidate mulligan counts are first filtered by **Pareto dominance**: a count k is discarded if some other count k' is at least as good on both axes and strictly better on at least one. This filtering step is free of any subjective weighting assumption, and in many cases collapses the decision to a single non-dominated choice on its own.

11.3 Epsilon-Lexicographic Tie-Breaking

When more than one candidate survives Pareto filtering — a genuine trade-off between the two criteria — the engine falls back to a lexicographic rule with tolerance, in the tradition of Herbert Simon's *satisficing* and Gigerenzer's *Take-The-Best* heuristic: a primary criterion is chosen (P(improvement), by default, on the reasoning that an ordinal quality score supports a robustness-style criterion better than it supports a raw expectation), and any candidate within a small relative tolerance of the best value on that primary criterion is retained; the final choice among that surviving set is then whichever maximizes the *secondary* criterion (EV). The tolerance band exists specifically to avoid a fragile decision driven by noise-level differences on the primary criterion alone, at the cost of a small, explicitly bounded concession on optimality.

12. Interactive Dashboard

A lightweight [Streamlit](#) application exposes the mined library as a searchable, filterable data table — the simplest possible interface that still meaningfully closes the loop between "a JSON file was produced" and "a person can actually browse and sanity-check it" without opening a text editor. Loading the mined JSON into a pandas DataFrame gives, for free, a full-text search box over card names and a live row count, and the same DataFrame is the natural staging point for the additional aggregate charts (parameter distributions by faction or type) that are the obvious next iteration of this tool, noted in the roadmap below.

13. Project Roadmap

The project is tracked as a living Kanban board (originally maintained in Obsidian); Figure 10 reproduces its current state. Items already in the **Done** column correspond directly to the components documented in this report; the **Backlog** and **To Do** columns outline the planned second phase of the project — large-scale statistical analysis of publicly shared decklists, a learned language-model component for deck-building assistance, and a formal game-theoretic treatment of resource allocation — which will be the subject of a follow-up report.

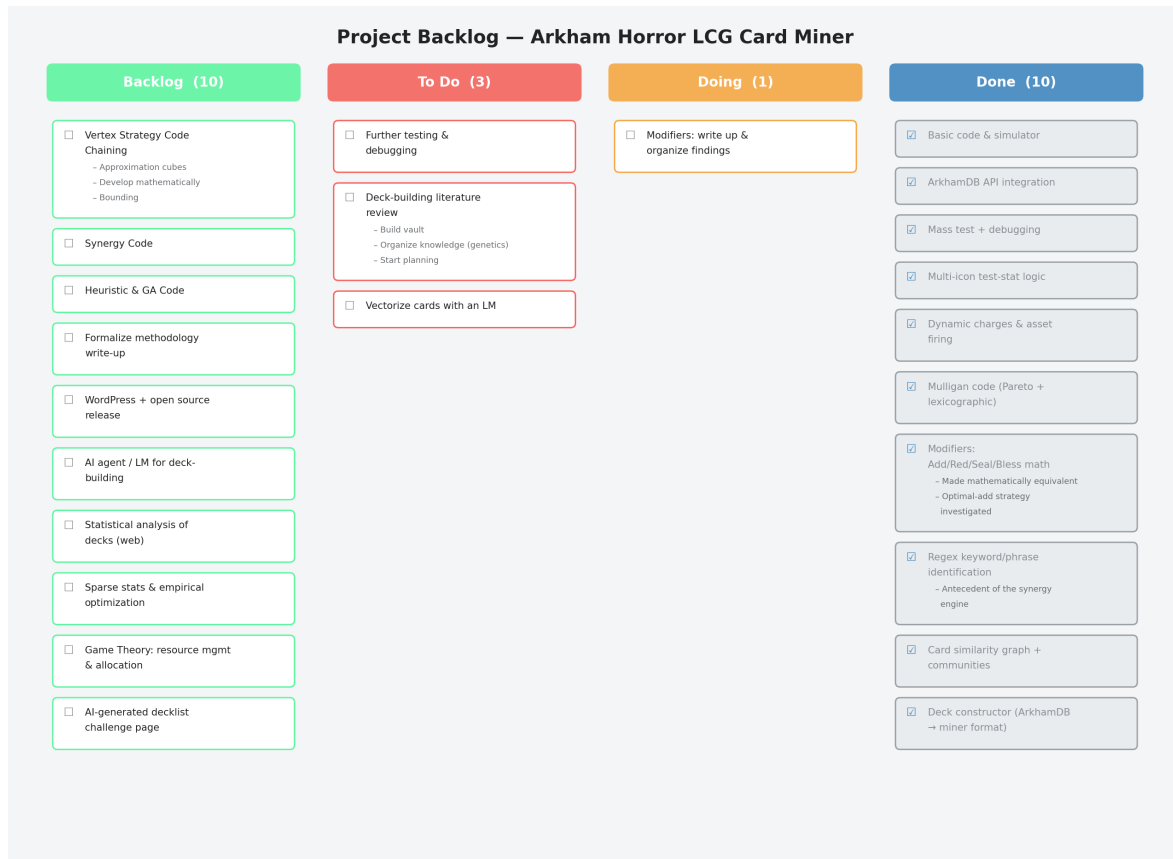


Figure 10. Project Kanban board, reconstructed from the author's Obsidian vault.

13.1 Phase Two: Statistical Validation of Deck-Building Hypotheses

The natural continuation of this work, briefly outlined here, is to turn the simulator's theoretical predictions into empirically testable claims against the population of decklists publicly shared on ArkhamDB. Several concrete, falsifiable hypotheses are already on record from the project's working notes and are the intended first targets: that smaller decks outperform larger ones; that a lower resource curve confers an advantage independent of raw card power; that aggressive mulliganing is favourable more often than intuition suggests; that pure card-draw effects are weak unless bundled with a cheap cost or a secondary effect; and that support-oriented builds outperform pure damage builds once the full scenario (not just combat) is accounted for. The planned toolset for this phase mirrors the same principle used throughout this report of matching the method to the structure of the question: association-rule mining (market-basket-style co-occurrence analysis) over real decklists as a data-driven complement to the text-based synergy signal of Section 4; deck-level regression against outcome proxies such as community ratings or campaign-completion reports; survival analysis for time-to-completion as a genuine time-to-event variable; the exact combinatorial mulligan machinery of Section 11 extended to a larger sample of real opening hands; and, for the specific question of resource timing (spend now versus bank for later, under a slowly-replenishing budget), a formal treatment of the deck as a **restless multi-armed bandit**, drawing on the same family of ideas as inventory theory and economic consumption-smoothing models. The remaining open backlog items — a fully data-driven *Synergy Code* built on top of the Section 4/5 machinery, and a learned language-model component for text-to-parameter extraction (Section 8 of the project's earlier design discussions) — are intended to feed into and be validated by this same statistical phase, rather than developed in isolation from it.

14. Discussion and Limitations

In the interest of the same regression-first, honest-about-edge-cases discipline used to validate the miner itself (Section 3.4), the following known limitations are stated explicitly rather than left implicit:

- **The geometric-clock charge approximation** (Section 8.4) is the model's one deliberate departure from an exact representation of the rules, made specifically to preserve the Markov property; its error is diluted, not eliminated, by keeping the simulated horizon realistic, and should be treated as an explicit modelling assumption whenever the simulator's output is quoted.
- **Name-keyed node identity** in the similarity graph (Section 5.3) silently merges distinct printings/experience-levels of a card that happen to share a display name; a production version of this analysis should key graph nodes on the unique ArkhamDB card code instead, retaining the display name only as a label.
- **The regex rule engine**, however heavily regression-tested, is fundamentally a hand-maintained grammar and will require ongoing extension as new card wordings are printed; Section 4.3's skip-gram discovery pipeline is intended precisely to make this extension process partly data-driven rather than purely reactive.
- **The alpha calibration** (Section 8.7) is fit against a single scalar fitness function; a natural extension is to calibrate against observed outcomes from real logged playthroughs once a sufficient sample is available, which is one of the motivations for the planned statistical-analysis phase in the roadmap (Section 13).

15. Conclusion

This report has documented a complete, working pipeline that takes an unstructured, publicly available natural-language card corpus and, through a sequence of well-established quantitative techniques — regex-based feature engineering, n-gram and skip-gram lexical statistics, vector-space embedding and graph community detection, directed-graph network analysis, closed-form recursive probability modelling, discrete-time Markov-chain simulation, Bayesian optimization, and multi-criteria decision theory — produces calibrated, interpretable, and visually inspectable answers to genuinely open questions about deck design. Every component was built and validated against real data (a corpus of 1,276 mined cards and a real example deck), not synthetic toy examples, and every method was chosen because its assumptions were judged to fit the specific sub-problem at hand — exact combinatorial enumeration where the state space allowed it, a Markov relaxation where it did not, Pareto filtering before any subjective weighting, and explicit, stated caveats wherever a modelling assumption departs from the true underlying system. The planned second phase (large-scale statistical analysis of real decklists, and a learned language-model component) is designed to build directly on top of the schema and infrastructure documented here.

Read as a single body of work rather than as a collection of separate scripts, the project is best understood as a self-contained rehearsal of the quantitative-analyst workflow end to end: acquiring and cleaning a messy real-world data source; engineering features from it with an explicit, testable, and regression-validated rule set rather than an opaque black box; discovering structure in the resulting dataset without supervision, through both lexical statistics and graph-theoretic community detection; building an explicit stochastic model of the underlying dynamical system and calibrating its free parameters against data; and, throughout, being disciplined about stating which modelling assumptions are exact and which are deliberate, bounded approximations. These are precisely the habits — not any single tool or library — that transfer directly to a professional quantitative research, data analysis, or applied data science role: the dataset here happens to be a card game, but the underlying workflow of turning unstructured information into calibrated, defensible, and honestly-caveated numerical answers is domain-agnostic. This project is intended as evidence of that workflow, and of the intention to keep extending it — the statistical validation phase outlined in Section 13.1 is

the next concrete step in that direction.

Appendix A: Repository and Code Structure

The codebase is organized into the following independently runnable modules:

Module	Role
Card Miner (Section 3)	ArkhamDB → structured JSON parameter schema.
Lexical Analysis (Section 4)	Unigram / n-gram / skip-gram frequency mining over the mined corpus.
Graph Analysis (Section 5)	Feature vectorization, k-NN similarity graph, Louvain communities, static and interactive rendering.
Gephi Prototype (Section 6)	Directed weighted graph export (CSV) for static network analysis.
Deck Constructor (Section 7)	ArkhamDB decklist -> simulator-ready records.
Markov Simulator (Section 8–9)	Per-action transition-matrix simulation, Bayesian-optimized calibration, diagnostic charts.
Chaos Bag Model (Section 10)	Closed-form recursive Bless/Curse success-probability model.
Mulligan Engine (Section 11)	Exact combinatorial EV / P(improvement), Pareto + lexicographic decision rule.
Dashboard (Section 12)	Streamlit library browser.

Repository: <https://github.com/justanotheruseeeeeer>